

TECH STACK & API KEYS

1. Stack (Phase 1 — locked, see memory.md before changing)

Layer	Choice	Notes
Backend	Python 3.11+, Flask	REST API, JWT auth
ML	scikit-learn (Logistic Regression)	<code>pandas</code> , <code>numpy</code> for features
Database	SQLite	file-based, zero setup; <code>backend/db/schema.sql</code>
Frontend	React (Vite) or plain HTML/CSS/JS	pick one in kickoff meeting, don't mix
Auth	PyJWT	tokens signed with <code>JWT_SECRET_KEY</code> env var
Notifications	Twilio (SMS sandbox) + SMTP (email)	see keys below
Dev tools	Git/GitHub, VS Code, Postman/Thunder Client	
Testing	pytest (backend), Jest (if React)	
Deployment (optional, Phase 1 demo)	Render / Railway / local	free tier is enough

Report mentions MySQL under "software requirements" but SQLite under "objectives/methodology."

Team decision: SQLite for Phase 1 (zero-setup, matches the persistence-layer description).

Record any change to this in `memory.md`.

2. API keys / external services — who needs what

None of these are required to get the core rules+ML scoring loop working. They're only needed for the notification module (Member D). Everything else runs with **zero external API keys**.

Service	Used for	Get a key	Env var
Twilio (free trial)	Sandbox SMS OTP / alert delivery	https://www.twilio.com/try-twilio	TWILIO_ACCOUNT_SID , TWILIO_AUTH_TOKEN , TWILIO_FROM_NUMBER
Gmail App Password / SMTP, or SendGrid free tier	Email alerts to analyst	Gmail: generate an App Password on the account; SendGrid: https://sendgrid.com	SMTP_HOST , SMTP_USER , SMTP_PASS (or SENDGRID_API_KEY)
ipapi.co or ipinfo.io (free tier)	Optional: resolve "location mismatch" from IP for the rule engine	https://ipapi.co / https://ipinfo.io	GEO_API_KEY (optional – code should fall back to a stub if unset)
– (self- generated)	JWT signing	Generate locally: <code>python -c "import secrets; print(secrets.token_hex(32))"</code>	JWT_SECRET_KEY

Nobody uses a real bank or telecom API in Phase 1 — that's explicitly future scope per the report. Don't request or wire up carrier SIM-change webhooks; `simulators/` fakes that event.

3. `.env.example` (commit this; never commit `.env`)

```
# --- core ---
JWT_SECRET_KEY=changeme_generate_locally
DATABASE_URL=sqlite:///backend/db/simswap.db

# --- notifications (Member D) ---
TWILIO_ACCOUNT_SID=
TWILIO_AUTH_TOKEN=
TWILIO_FROM_NUMBER=
```

```
SMTP_HOST=
SMTP_USER=
SMTP_PASS=

# --- optional ---
GEO_API_KEY=
```

4. Setup order for the team (day 1)

1. Everyone clones the repo, creates `.env` from `.env.example` (only Member D fills in real sandbox keys — others can leave notification vars blank, the notifier should no-op safely).
2. `pip install -r requirements.txt` (backend), `npm install` (frontend, if React).
3. Member D runs `backend/db/schema.sql` to create the local SQLite file and commits a seed script — everyone else works against the same schema from day 1.
4. Member B stubs `risk_engine` functions with fixed return values so Member A can build API routes against a real function signature immediately, before the model is trained.